Reviewer Pack — Permanent of Sensitive-Boundary Bipartite Matrix Bounds Lift...

Entry 25722288cd28 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 10:47:08 UTC

Permanent of Sensitive-Boundary Bipartite Matrix Bounds Lifted Indexing CC

Entry ID: `25722288cd28` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 10:47:08 UTC

1. Conjecture

Field A (mathematical branch): Permanents of bipartite 0/1 biadjacency matrices via Ryser inclusion-exclusion enumeration of perfect matchings (algebraic combinatorics on the Boolean cube; rarely deployed in decision-tree / communication lower bounds) **Field B** (complexity object): Deterministic communication complexity D^cc(f ∘ IND_b) of Indexing-lifted Boolean functions, accessed through decision-tree depth Q^dt(f) via the Raz-McKenzie / Göös-Pitassi-Watson lifting theorem

Statement:

> For every non-constant Boolean function f: {0,1}^k → {0,1} with k ≤ 4, let B(f) be the 2^{k-1} × 2^{k-1} biadjacency matrix of the sensitive boundary subgraph of the Hamming cube Q_k between even-parity and odd-parity vertices, defined by B(f)_{x,y} = 1 iff |x ⊕ y| = 1 and f(x) ≠ f(y). Conjecture: log_2(perm(B(f)) + 1) ≤ k · Q^dt(f), where perm denotes the matrix permanent (counting perfect matchings of the sensitive boundary). By the GPW indexing lift this gives D^cc(f ∘ IND_b) ≥ Ω(log b · k^{-1} · log_2(perm(B(f)) + 1)), a permanent-based lower bound on lifted CC.

Rationale (proposer's reasoning):

> Sensitive edges form a bipartite subgraph of Q_k whose perfect-matching count is a global, non-spectral combinatorial invariant orthogonal to Fourier / influence-based measures already used in lifting arguments. Decision-tree depth controls the resolution of sensitive transitions: shallow trees can only sustain sensitive-edge structures supporting a limited number of disjoint perfect matchings, whereas parity-like deep functions saturate the bound. Because permanents are not spectral functionals, this route does not invoke the natural-proofs / algebrization barriers that block direct rank-style lifts.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f03ba3422400bb7c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across all 2^16 Boolean functions on k=4 (or, if sampled, ≥5000 uniform random truth tables plus all symmetric functions), verify log2(perm(B(f))+1) ≤ 4·Q^dt(f). Conjecture is SUPPORTED iff support_fraction = 1.0 (zero violators) and worst-case slack (4·Q^dt(f) − log2(perm(B(f))+1)) ≥ 0 for every f tested; FALSIFIED by any single f with slack < 0.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `permanent biadjacency matrix sensitive boundary hypercube decision tree complexity` - `Raz-McKenzie lifting theorem indexing gadget communication complexity permanent lower bound` - `perfect matchings sensitivity graph Boolean function query complexity Goos Pitassi Watson`

Top relevant hits considered: - [http://arxiv.org/abs/2512.11833v1] Soft Decision Tree classifier: explainable and extendable PyTorch implementation - [http://arxiv.org/abs/1810.08668v2] Parity Decision Tree Complexity is Greater Than Granularity - [http://arxiv.org/abs/2209.08042v1] Decision Tree Complexity versus Block Sensitivity and Degree - [http://arxiv.org/abs/1904.13056v4] Query-to-Communication Lifting Using Low-Discrepancy Gadgets - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size - [http://arxiv.org/abs/cs/990600 A Lower Bound on the Average-Case Complexity of Shellsort - [http://arxiv.org/abs/1703.07666v1] Query-to-Communication Lifting for BPP - [http://arxiv.org/abs/2306.14401v1] On the distribution of sensitivities of symmetric Boolean functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import sys
import random

def binomial_coefficient(n, k):
    if k > n:
        return 0
    result = 1
    for i in range(k):
        result *= (n - i)
        result //= (i + 1)
    return result
```

```
def matrix_multiplication(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def transpose_matrix(A):
    m = len(A)
    n = len(A[0])
    T = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            T[j][i] = A[i][j]
    return T

def permanent(B):
    n = len(B)
    if n == 1:
        return B[0][0]
    p = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in B[1:]]
        sign = (-1) ** j
        p += sign * B[0][j] * permanent(submatrix)
    return p

def sensitive_boundary_matrix(f, k=4):
    n = 2 ** (k - 1)
    B = [[0] * n for _ in range(n)]
    for x in range(n):
        for y in range(x + 1, n):
            if sum(int(a) != int(b) for a, b in zip(bin(f"{x:0{k-1}b}"), bin(f"{y:0{k-1}b}"))) ==
                B[x][y] = 1
    return B

def Q_dt(f):
    memo = {}

    def helper(subset):
        if not subset:
            return 0
        if tuple(sorted(subset)) in memo:
            return memo[tuple(sorted(subset))]

        min_val = float('inf')
        for i in range(len(subset)):
            new_subset = subset[:i] + subset[i+1:]
            max_val = -float('inf')
            for j in range(2):
                if j not in new_subset:
                    new_subset_with_j = new_subset + (j,)
```

```
                    max_val = max(max_val, helper(new_subset_with_j))
            min_val = min(min_val, max_val)

        memo[tuple(sorted(subset))] = min_val
        return min_val

    return helper(tuple(range(2**(k-1))))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    k = 4
    num_functions = 2 ** (k * k)
    total_slack = 0
    worst_case_slack = float('-inf')
    instances_tested = 0

    for _ in range(num_functions):
        f = {bin(i)[2:].zfill(k): random.choice([0, 1]) for i in range(2**k)}

        B = sensitive_boundary_matrix(f)
        perm_B = permanent(B)
        Q_dt_f = Q_dt(f)

        slack = 4 * Q_dt_f - (perm_B + 1).bit_length()
        total_slack += slack
        worst_case_slack = max(worst_case_slack, slack)
        instances_tested += 1

    mean_slack = total_slack / instances_tested
    support_fraction = 0.0 if worst_case_slack < 0 else 1.0

    return {
        "metric_name": "slack",
        "metric_value": mean_slack,
        "instances_tested": instances_tested,
        "conjecture_holds": support_fraction == 1.0,
        "counterexample": "" if worst_case_slack >= 0 else f"worst_case_slack={worst_case_slack}"
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_slack = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_slack} std=0.0 support_fraction={support_fraction}")
    elif any(r["counterexample"] for r in results):
        first_failing_seed = next(s for s, r in enumerate(results) if r["counterexample"])
```

```
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8dfed8b.py", line 128, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8dfed8b.py", line 103, in run_trial
    B = sensitive_boundary_matrix(f)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e8dfed8b.py", line 63, in sensitive_bou
      if sum(int(a) != int(b) for a, b in zip(bin(f"{x:0{k-1}b}"), bin(f"{y:0{k-
1}b}"))) == 1 and f[x] != f[y]:
                                                ^^^^^^^^^^^^^^^^^^^
TypeError: 'str' object cannot be interpreted as an integer
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a TypeError in sensitive_boundary_matrix (misuse of bin() on a formatted string) before producing any data, so the pre-registered support condition cannot be evaluated. | next: Fix the bit-comparison bug (e.g., use Hamming distance via bin(x ^ y).count('1') on integers x,y in range(2^k)) and re-run the full 2^16 enumeration on k=4.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	413429
2	preregistration	claude_max	opus	0	0	7545
3	novelty	claude_max	opus	0	0	3644
4	novelty	claude_max	opus	0	0	11672
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11111
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13075
7	judge	claude_max	opus	0	0	4482

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 464958 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/25722288cd28.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/25722288cd28.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/25722288cd28.tar.gz` (if generated)